Efficient Multi-Objective Optimization using 2-Population Cooperative Coevolution

EUROCAST 2013 - Las Palmas de Gran Canaria, Spain

Alexandru-Ciprian Zăvoianu[1,3], Edwin Lughofer[1], Wolfgang Amrhein[2,3], Erich Peter Klement[1,3]

[1]Department of Knowledge-based Mathematical Systems/Fuzzy Logic Laboratory
Linz-Hagenberg, Johannes Kepler University of Linz, Austria

[2]Institute for Electrical Drives and Power Electronics,
Johannes Kepler University of Linz, Austria

[3]Austrian Center of Competence in Mechatronics, Linz, Austria

Outline

Modern Prototyping of Electrical Drives

Our **current research efforts** w.r.t. evolutionary computation are **primarily motivated** by practical applications from the field of **electrical drive engineering**.

The task of **designing and prototyping electrical drives** is a **multi-objective optimization process** that aims to find machines that simultaneously have:

- a high energy efficiency
- very good fault tolerance
- easily controllable operational characteristics
- a low manufacturing price

Modern Prototyping of Electrical Drives

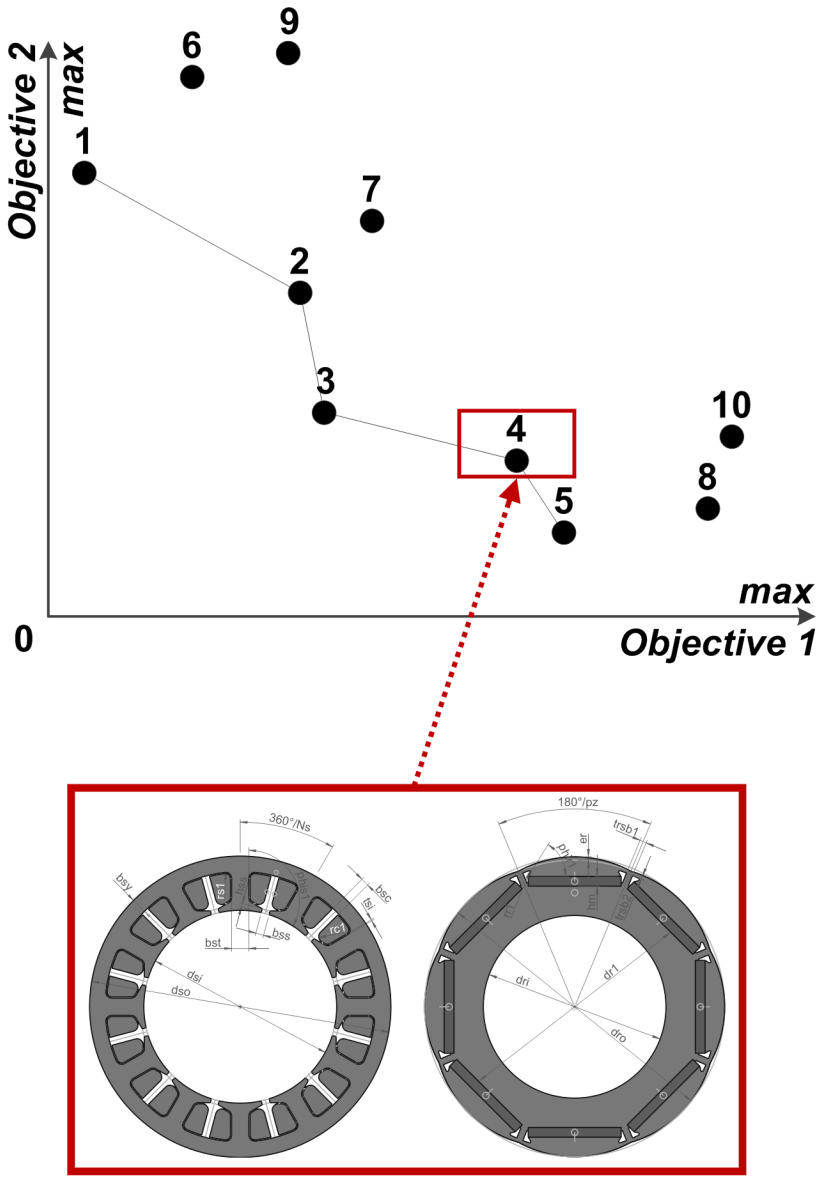
In spite of using **modern search approaches** like response surface methods, particle swarm optimization and evolutionary algorithms, the **prototyping process is very slow** (one optimization run can take several days) even when distributing the computations over a computer cluster.

The reason is that the fitness evaluation functions rely on very time intensive (e.g., 5 to 10 minutes) **finite element simulations**.

As there is a strong push towards **improving the overall speed of the design and prototyping process**, the development of efficient multi-objective optimization methods is of great importance.

The result of a multi-objective optimization problem (MOOP) is usually a set of **Pareto non-dominated solutions** that is called the **Pareto front**.

In our case, **each solution codifies an electrical drive design** that might prove interesting to a commercial partner



Multi-Objective Evolutionary Algorithms

Evolutionary Algorithms work by generation-wise improving sets (populations) of solutions.

Extensions aimed at making them discover **multiple Pareto optimal solutions** in a **single run** were natural and proved very successful.

Two of the best performing and most widely used **early multi-objective evolutionary algorithms** (MOEAs) are **NSGA-II** [1] and **SPEA2** [5].

Two **specialized genetic operators** are generally recommended to be used within NSGA-II and SPEA2:

- **Simulated Binary Crossover** (SBX)
- **Polynomial Mutation**

Both operators can be tuned to produce offspring that are closer or further away from their parents.

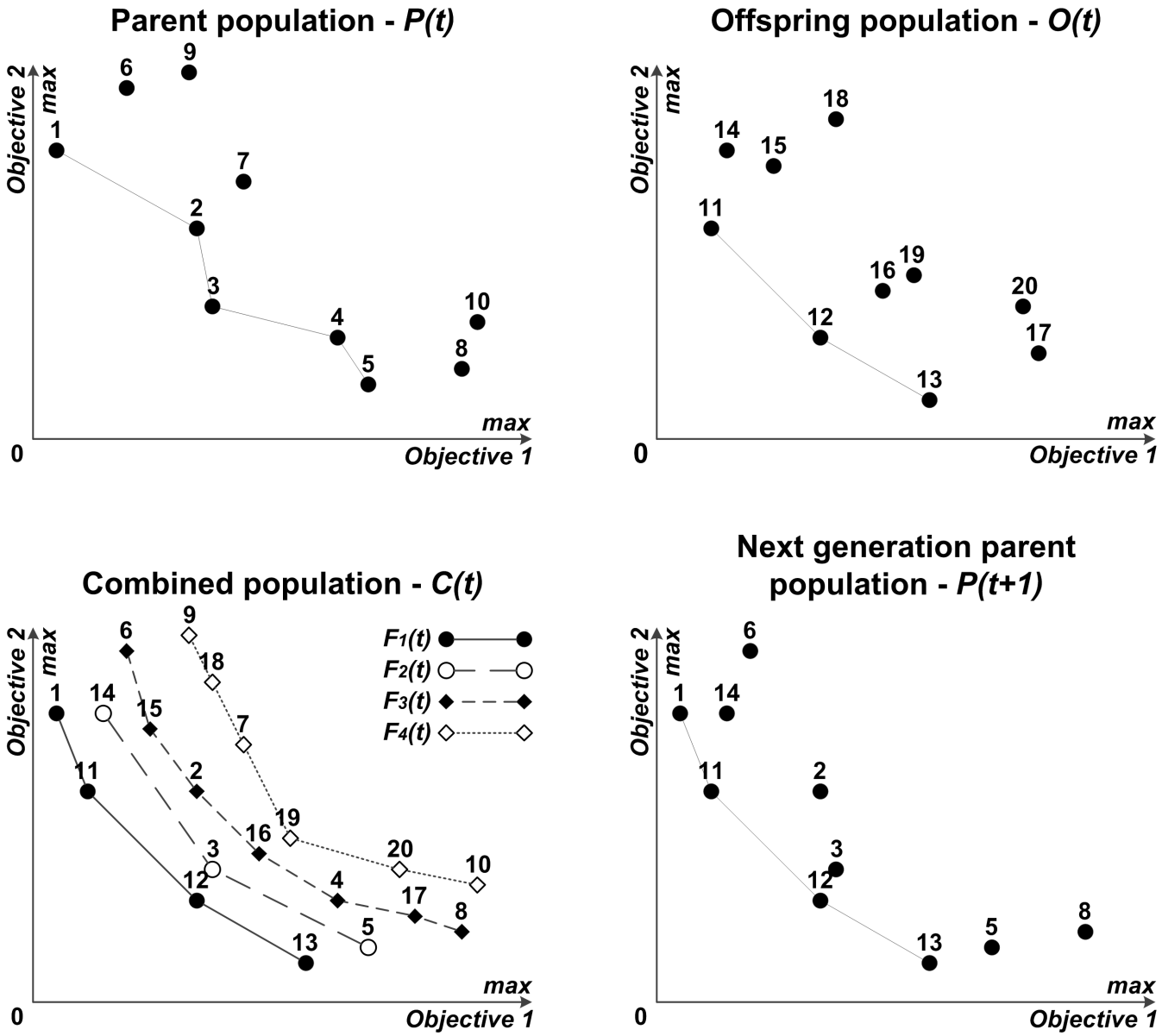
Multi-Objective Evolutionary Algorithms

Both **NSGA-II and SPEA2** are based on the **same** two major **design principles**:

- a **highly elitist approach to evolution** implemented using a secondary population
- a **two-tier fitness evaluation function** (**non-dominated sorting** for NSGA-II and **environmental selection** for SPEA2) based on:
  - a primary Pareto non-dominance metric
  - a secondary search space crowding metric

In fact, the two algorithms are implementations of the
$(\mu + \alpha)$ - **EA strategy** orientated towards multi-objective optimization

Multi-Objective Evolutionary Algorithms



Differential Evolution Operators

Differential evolution (DE) operators were introduced by Storn and Price [3].

They are **especially suitable for continuous optimization problems**, although some discrete-value versions have been proposed.

They are based on very simple concepts but can **efficiently explore very complex search spaces**.

The DE operators **have become widely used** and are integrated in several **state-of-the-art single objective optimization algorithms** (e.g., CoDE, EPSDE)

Differential Evolution Operators

An offspring solution (vector) **y** is generated by performing crossover on two parents:

- parent (vector) **x** - directly chosen from the current population
- parent (vector) **v** - a mutant vector that can be constructed according to different mutation strategies

Several mutation/crossover combinations have been proposed.

Usually, in single objective optimization contexts, the fittest individual between **x** and **y** will be included in the population of the next generation.

Differential Evolution Operators

Common DE mutation types:

- **rand/1**

$$\mathbf{v} = r_1 + F(r_2 - r_3) \tag{1}$$

- **rand/2**

$$\mathbf{v} = r_1 + F(r_2 - r_3) + F(r_4 - r_5) \tag{2}$$

- **current-to-rand/1**

$$\mathbf{y} = r_1 + U(r_1 - \mathbf{x}) + F(r_2 - r_3), \tag{3}$$

where $r_i \neq \mathbf{x}, i = \overline{1,5}$ are different, randomly chosen individuals from the current population, $F > 0$ is a control parameter and U is a uniformly distributed random value in $[0, 1]$.

Differential Evolution Operators

Common DE crossover types:

- **binomial**

$$\mathbf{y}[j] = \begin{cases} \mathbf{v}[j], & \text{if } U_j \leq CR \text{ or } j = I \\ \mathbf{x}[j], & \text{if } U_j > CR \text{ or } j \neq I \end{cases}, \tag{4}$$

  where CR in $[0, 1]$ is a control parameter, $I \leq d$ is a randomly chosen integer and U_j is an independent random variable uniformly distributed in $[0, 1)$.

- **polynomial** – the starting position k of the crossover process is chosen randomly and the next consecutive L elements (counted in circular manner) are chosen from $\mathbf{v}$ with a probability that decreases exponentially with an increased distance from k.

DE-based MOEAs

Early attempts to integrate DE operators in multi-objective evolutionary algorithms have also **proven quite successful**: the **DEMO** [4] and **GDE3** [2] algorithms.

The **integration of DE** in these algorithms is quite **straight forward** as the **SBX/Polynomial Mutation** operator combination **was replaced by** a **DE/rand/1/bin** operator (i.e., a combination of (1) and (4)) **while preserving** the **MOEA specific elitist selection** mechanism:

- non-dominated sorting for NSGA-II
- environmental selection for SPEA2

DE-based MOEAs

As DE appears to explore the search space more efficiently, **DEMO and GDE3 generally converge much faster than NSGA-II and SPEA2**.

But there are also two important downsides:

- the **DE-operators** seem **less robust than SBX/Polynomial Mutation w.r.t.** their **parameterization** (e.g., the DEMO recommended values of $F = 0.5$ and $CR = 0.3$ are not the best choice for several test problems)
- on some problems, **NSGA-II and SPEA2** significantly **outperform DEMO and GDE3**

Outline

Current Research Focus

Develop an optimization method that **effectively combines** the **general advantage in convergence time induced by DE with** the overall **more stable behavior exhibited by the SBX/Polynomial Mutation** operator combination.

Ideas

Our initial attempts centered around the idea of having a pool with different operators (various DE and SBX configurations) that the MOEA could choose from **did not prove very successful**.

Coevolution was the next **promising candidate** as this technique can easily be adapted in order to **smoothly integrate multiple search space exploration behaviors into a single evolutionary run**.

Coevolution

In biology **coevolution** designates the notion of **different species acting as foils against one another and causing one another to adapt (and improve)**. In algorithm design, this is the idea behind **competitive coevolution**.

Coevolution also includes **symbiotic** (i.e., mutually beneficial) **relationships**. In algorithm design, this translates as **cooperative coevolution**.

In the field of **soft computing**, **coevolution** is usually applied to population based optimization methods and **is implemented using subpopulations that are evolved simultaneously and share fitness information**.

N-Population Cooperative Coevolution

It is primarily used in order to **break up complicated high-dimensional search spaces into N**, much simpler, **subspaces** that are **explored by independent populations**.

The particular way in which **we apply cooperative coevolution** does not implement the search space partitioning concept, but, instead **takes full advantage of two general (complementary) characteristics of the method**:

- it helps to **maintain diversity** in the evolutionary system
- it enables the **rapid dissemination of breakthrough solutions**

A 2-Pop. Cooperative Coevolution MOEA

Simultaneously evolve two distinct populations of equal sizes:

- Population **P** relies on a **DE/rand/1/bin** operator to evolve new individuals ($F = 0.5$, $CR = 0.2$)
- Population **Q** relies on a **SBX/Polynomial Mutation operator combination** (standard parameter settings):
  - SBX with a `crossover probability` = 0.9 and a `crossover distribution index` = 20
  - Poly. Mutation with a `mutation probability` = $1/d$ (d is size of the solution vector) and a `mutation distribution index` = 20

The **SPEA2 environmental selection strategy** is applied on both populations as a **crowding survival operator**.

A 2-Pop. Cooperative Coevolution MOEA

Fitness is **shared** by simply **passing individuals between** the two **populations**.

After experimenting with different strategies, we discovered that a **dual fitness sharing mechanism** **displays the best performance**.

The fitness sharing consists of:

- **weak sharing stages** - occur at every generation
- **strong sharing stages** - occur at every $\mathbf{m^{th}}$ generation (during our experiments so far, we found that the setting $\mathbf{m} = |\mathbf{P}|/10$ is stable and generally yields good results)

A 2-Pop. Cooperative Coevolution MOEA

The **weak sharing stage**:

- **randomly select one individual from** the offspring population of **P and one from** the offspring population of **Q**
- **try to introduce each individual in the complementary population via environmental selection**

The **strong sharing stage**:

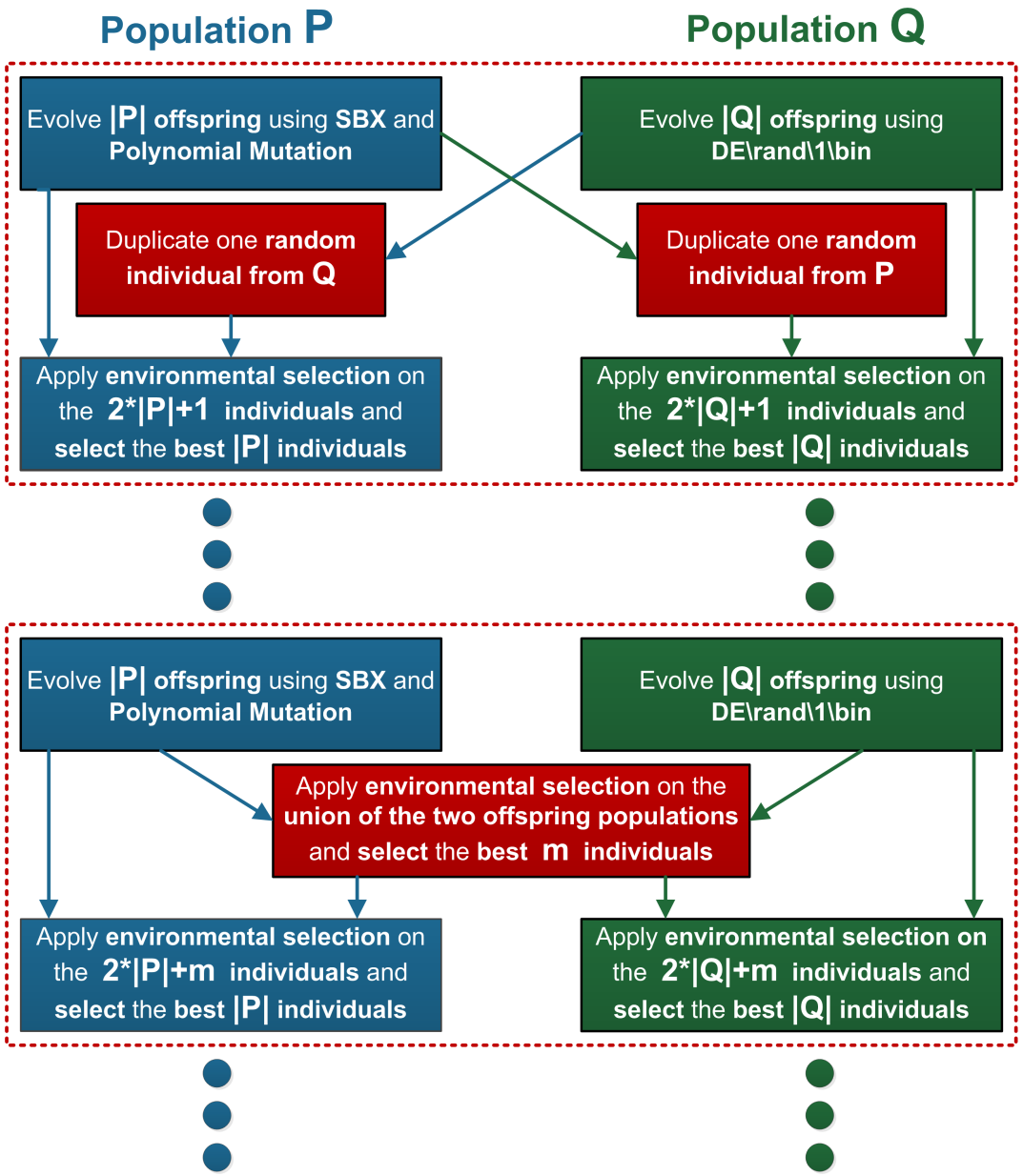
- **construct the union of** the offspring populations of **P and Q** and **extract via environmental selection the best m individuals from the combined population**
- **try to introduce the m elite individuals in populations P and Q via environmental selection**

A 2-Pop. Cooperative Coevolution MOEA

We make **extensive use of the environmental selection operator** because it inherently has two features that are enhancing the robustness of the coevolution process:

- **a crowding effect** - it filters out individuals that are very similar to other individuals (w.r.t. the objective functions)
- **a diversity preservation effect** - it selects individuals that maintain an accurate, well spread out, approximation of the Pareto front

A 2-Pop. Cooperative Coevolution MOEA



1st generation:
weak fitness sharing stage

mth generation:
strong fitness sharing stage

Outline

Performance Tests

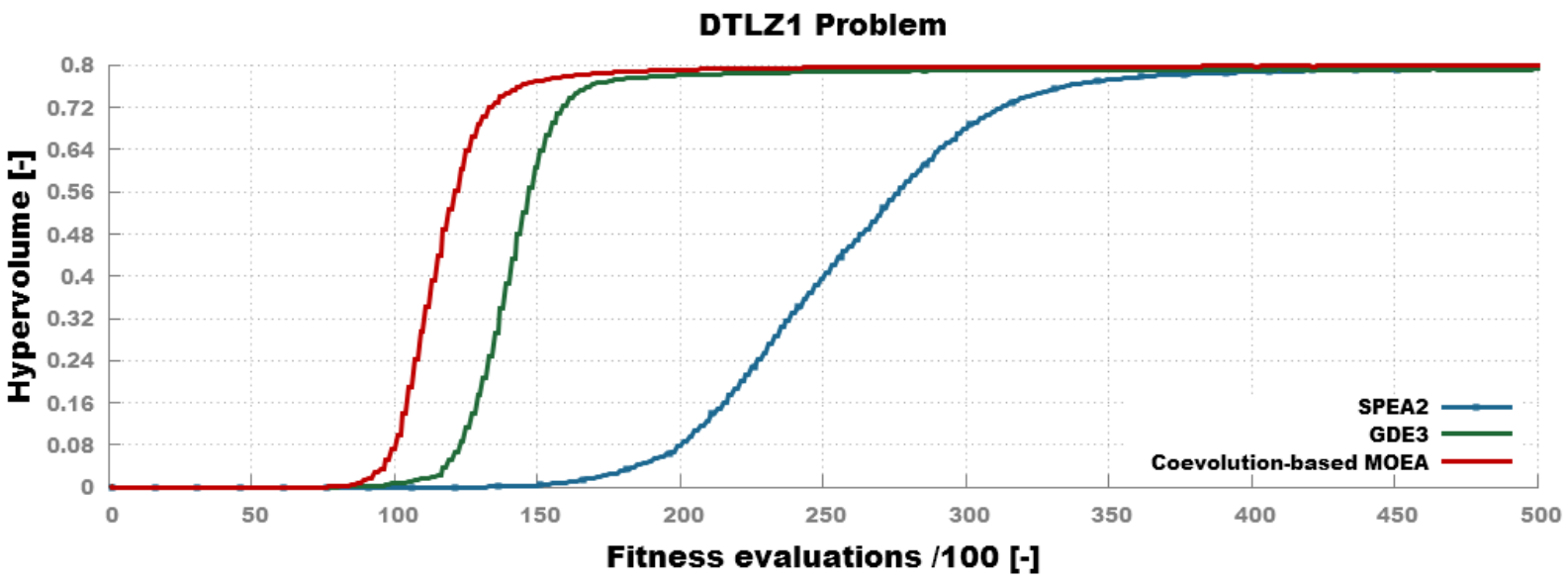
We performed tests on **10 benchmark MOOPs** using:

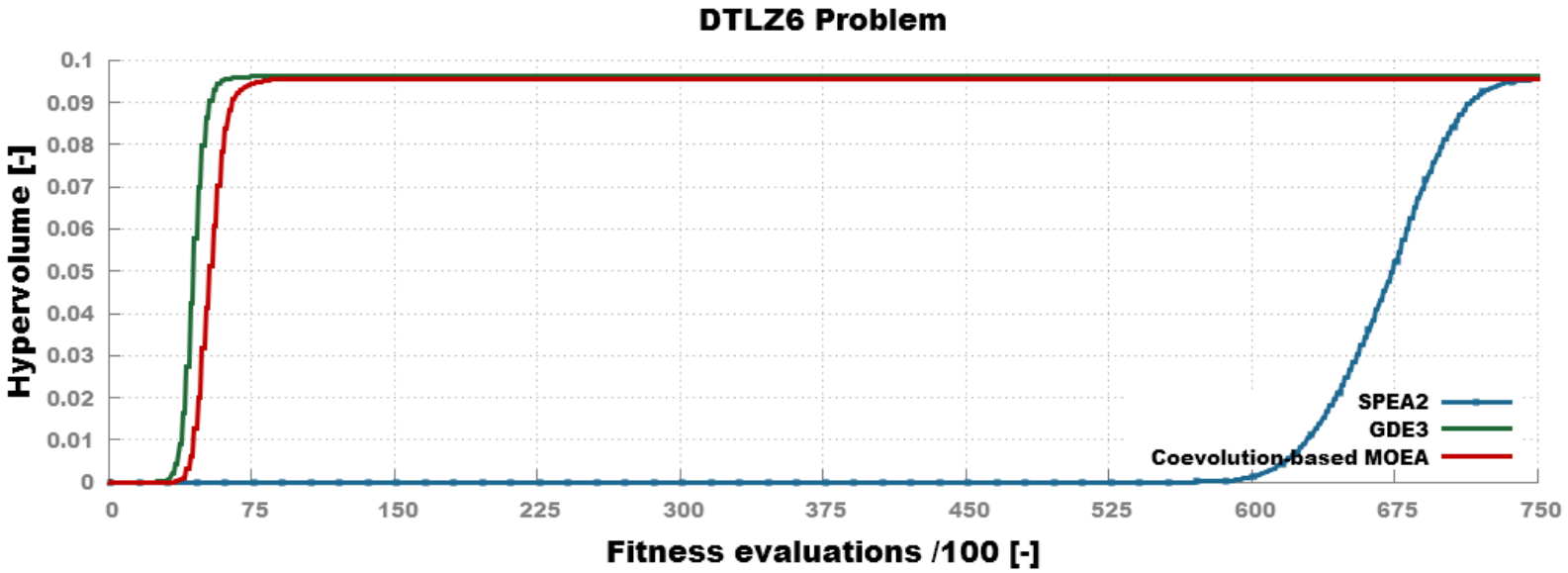
- **NSGA-II/SPEA2** (standard parameter settings, population and archive size of 200)
- **DEMO/GDE3** (standard parameter settings , population size of 200)
- **the 2-population cooperative coevolution MOEA** (the size of both **P** and **Q** is set to 100)

Each **evolutionary run** was stopped after **50.000 fitness evaluations** (and after 75.000 evaluations in two special cases).

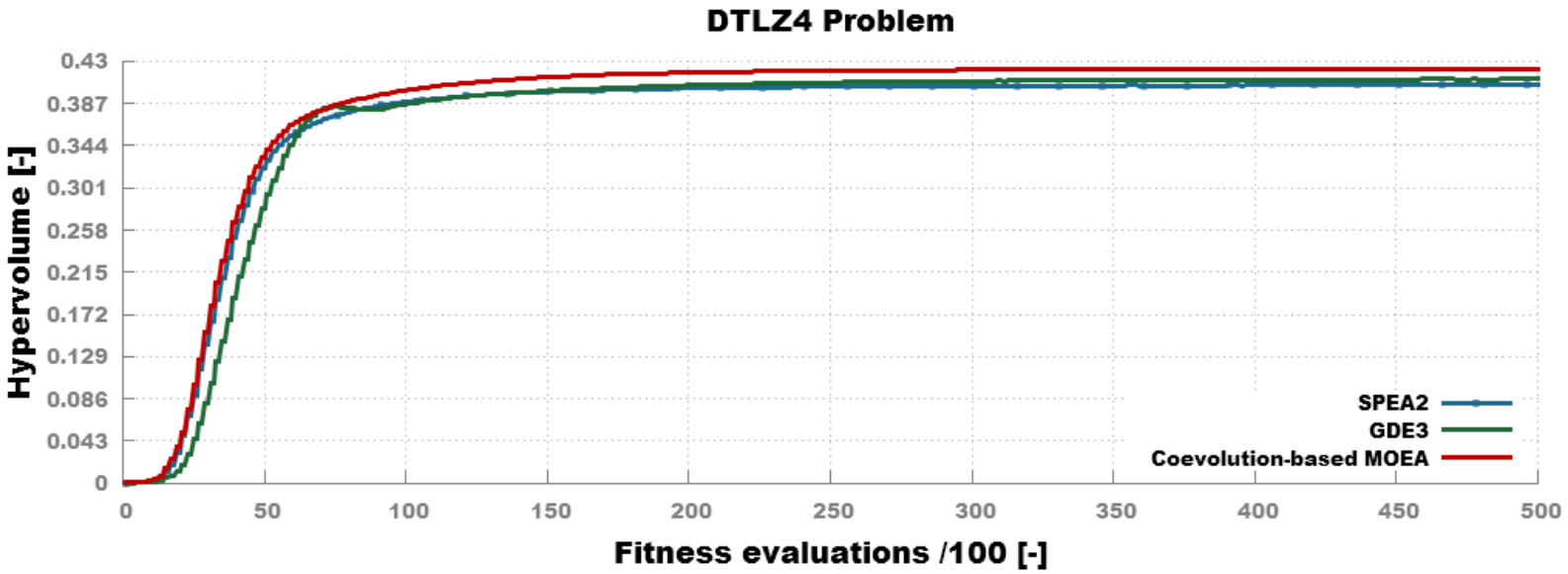
We **repeated each test 100 times** and we present averaged results.

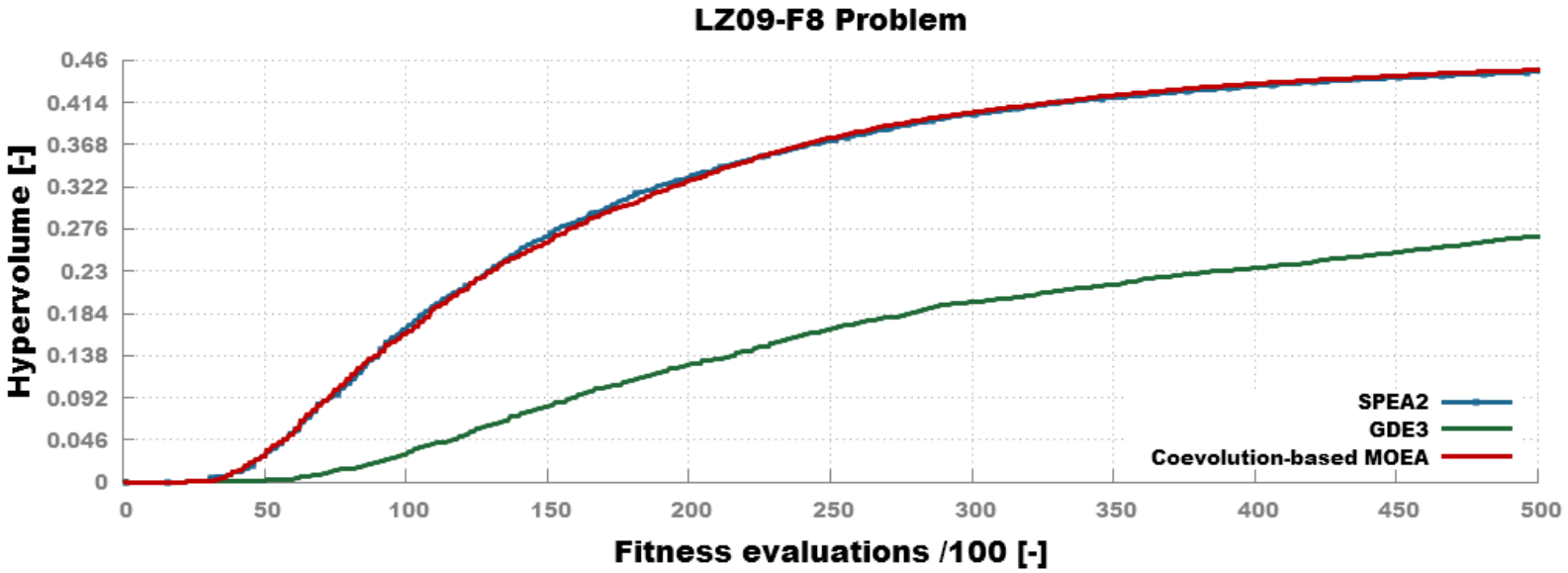
Results - Artificial Test Problems



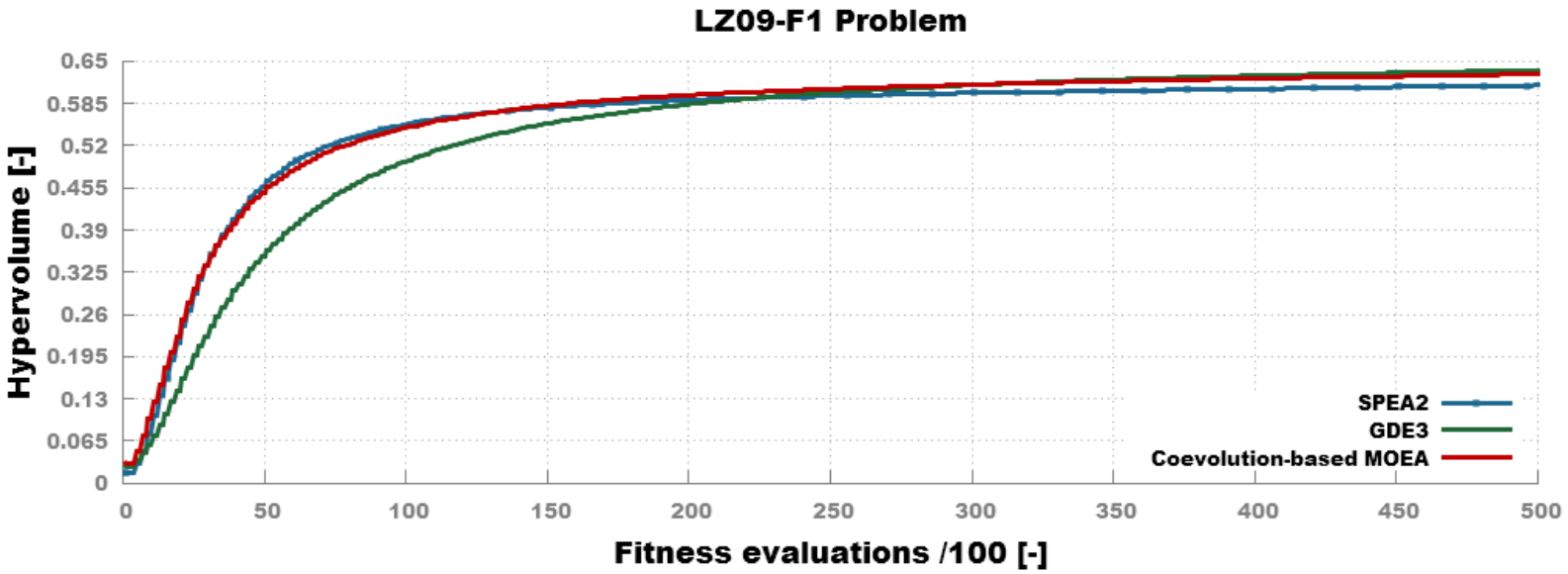


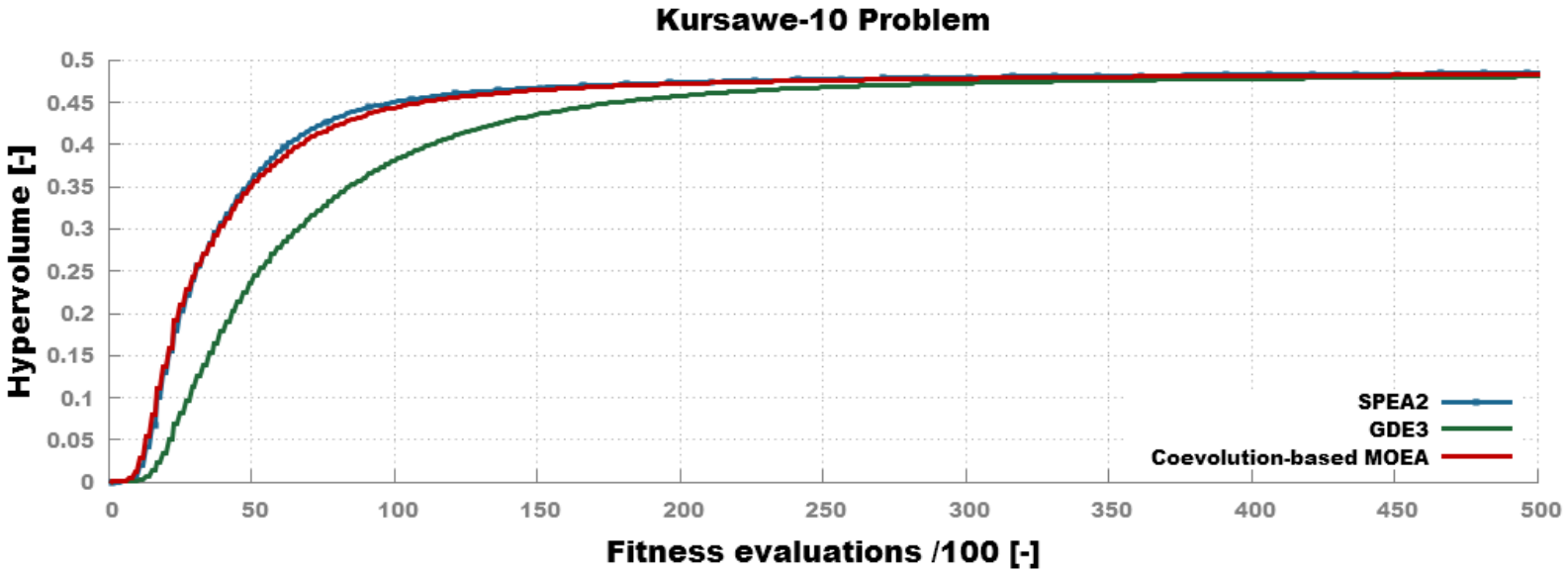
Results - Artificial Test Problems



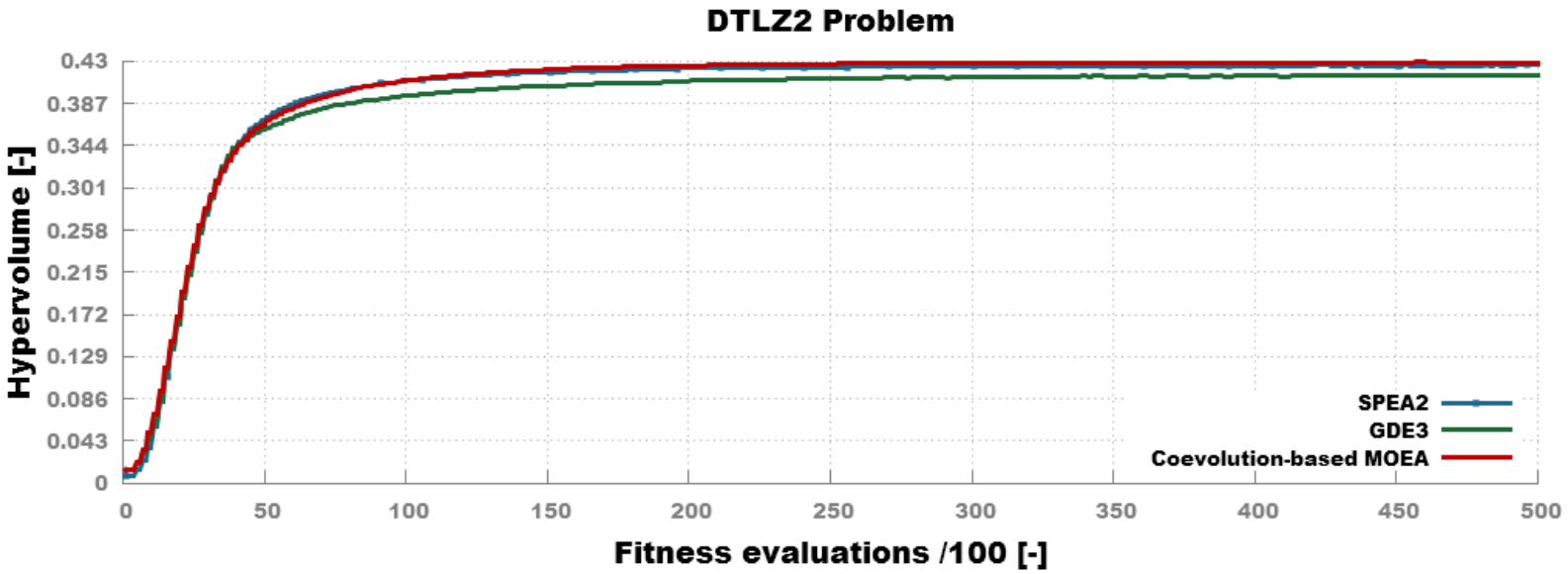


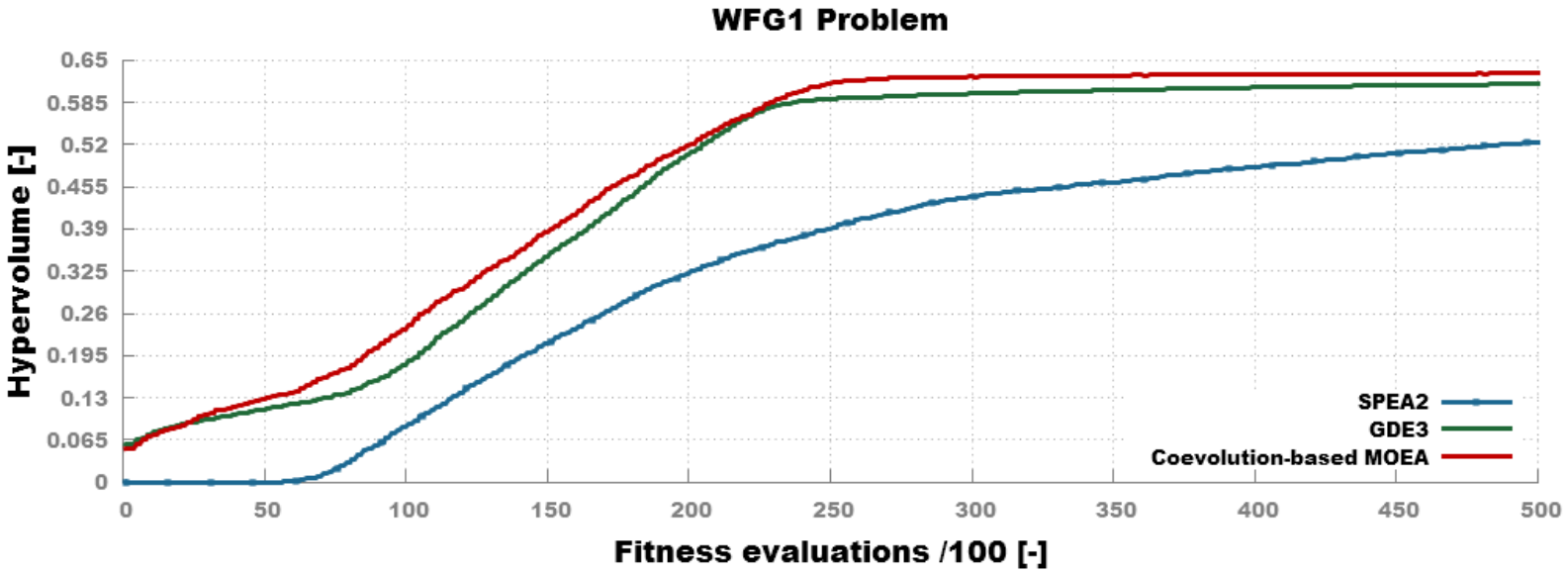
Results - Artificial Test Problems





Results - Artificial Test Problems





Outline

Conclusions

The obtained results indicate that **coevolution** is quite **successful at** its aim of **constructing a robust average between SBX/Polynomial Mutation and DE based methods** in the context of MOEA development:

- our algorithm generally converges faster than NSGA-II and SPEA2
- it also seems more stable than DEMO and GDE3

The **proposed fitness sharing mechanism introduces only one extra configuration parameter**, namely **m** - the control parameter of the strong fitness sharing stage.

Future Work

Back-up the conclusions with a **statistical analysis test**.

Integrate the 2-population cooperative coevolution MOEA **into the electrical drive optimization framework** developed at the **Austrian Center of Competence in Mechatronics**.

Try to **develop** an **asynchronous variant** of the algorithm.

References

Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation 6(2), 182 –197 (2002)

Kukkonen, S., Lampinen, J.: GDE3: The third evolution step of generalized differential evolution. In: The 2005 IEEE Congress on Evolutionary Computation. vol. 1, pp. 443–450. IEEE (2005)

Price, K., Storn, R., Lampinen, J.: Differential evolution. Springer (1997)

Robič, T., Filipič, B.: Demo: Differential evolution for multiobjective optimization. In: Evolutionary Multi-Criterion Optimization. pp. 520–533. Springer (2005)

Zitzler, E., Laumanns, M., Thiele, L.: SPEA2: Improving the strength Pareto evolutionary algorithm for multiobjective optimization. In: Giannakoglou, K., et al. (eds.) Evolutionary Methods for Design, Optimisation and Control with Application to Industrial Problems (EUROGEN 2001). pp. 95–100. International Center for Numerical Methods in Engineering (CIMNE) (2002)

Thank you very much!